Nesne Yönelimli Programlama – Konu Anlatımı

Nesne Yönelimli Analiz

Slaytlar, ders kitabı (Schach 8e), ders videoları ve WhatsApp grubu temel alınmıştır

9 Kasım 2026

7 Nesne Yönelimli Analiz (Chapter 13)

! Hoca derste vurguladı Sınavdaki yeri

2014 vizesi: *"Class yapısını oluşturun. Class tiplerini belirtin (Entity, Boundary, Control class) ... Collaboration veya sequence diyagramlarından birini seçip diyagram oluşturunuz."* 2025–26 vizesinin son sorusu (24 puan): rent-a-car otomasyonunun **sequence diyagramı**. WhatsApp: *"Sequence'de oklar üzerindeki çağrılar okun ucundaki sınıfın bir metodu olmalıdır."*

7.1 Analiz iş akışı ve üç sınıf türü

Object-oriented analysis (OOA) nesne yönelimli paradigmanın yarı biçimsel (semiformal) analiz tekniğidir (60'tan fazla eşdeğer teknik var; bugün tek geçerli alternatif Unified Process'tir). Bu iş akışında **sınıflar çıkarılır**. Analiz iş akışının iki amacı: gereksinimleri **daha derin anlamak** ve onları **bakımı kolay bir tasarım ve gerçekleştirmeye** götürecek şekilde tanımlamak.



Entity class



Boundary class



Control class

Tür	Ne modeller?	Örnek
Entity class	**Uzun ömürlü (long-lived) bilgiyi**; gerçek sistemdeki varlıkları.	Account Class, Investment Class, Mortgage Class, Kişi, Araç
Boundary class	Ürün ile **çevresi arasındaki etkileşimi**; genellikle **giriş/çıkış** ile ilgilidir. Her giriş ekranı, çıkış ekranı ve rapor kendi boundary sınıfıyla modellenir.	User Interface Class, Investments Report Class, Mortgages Report Class
Control class	**Karmaşık hesaplamaları ve algoritmaları**. Her önemsiz olmayan hesaplama bir control sınıfıyla modellenir.	Estimate Funds for Week Class, Elevator Controller

Bu üç tür UML'in **stereotype** genişletmesidir (`«entity»`, `«boundary»`, `«control»`). Hoca tahtada: entity = altı çizili daire, boundary = sol tarafında dikey çizgi olan daire, control = üstünde ok olan daire.

7.2 Entity sınıflarını çıkarmanın üç adımı

Aşağıdaki üç adım **artırımlı ve yinelemeli** yapılır:

1. **Functional modeling (işlevsel modelleme):** bütün use case'lerin **senaryolarını** sun. (*Senaryo, bir use case'in bir örneğidir.*)
2. **Entity class modeling (sınıf modelleme):** entity sınıflarını ve niteliklerini (attributes) belirle; aralarındaki ilişkileri ve etkileşimleri belirle; bunu bir **class diagram** olarak sun.
3. **Dynamic modeling (dinamik modelleme):** her entity sınıfı tarafından veya ona yapılan işlemleri belirle; bunu bir **statechart** olarak sun.

7.3 Asansör problemi (elevator problem)

Bir binada m kat ve n asansör var. (1) Her asansörde her kat için bir tane, toplam m düğme vardır; basılınca yanar ve asansörü o kata götürür; asansör o kata gelince söner. (2) İlk ve en üst kat hariç her katta iki düğme vardır (yukarı / aşağı); basılınca yanar; asansör o kata gelip istenen yönde hareket edince söner. (3) İsteği olmayan asansör, kapıları kapalı olarak bulunduğu katta bekler.

7.3.1 Functional modeling

Use case, ürün ile aktörler arasındaki etkileşimi **genel olarak** anlatır. Asansör probleminde sadece iki use case vardır: **Press an Elevator Button** ve **Press a Floor Button** (aktör: User). Senaryo bir use case'in bir örneğidir; hedef ürünü kavramak için **yeterli sayıda** senaryo incelenmelidir (hoca: ne çok fazla ne çok az).

- **Normal senaryo:** User A 3. katta yukarı düğmesine basar, 7. kata gitmek ister; düğme yanar; asansör gelir (içinde 1. katta binip 9. kat düğmesine basmış User B vardır); yukarı düğmesi söner, kapılar açılır, User A biner, 7. kat düğmesine basar...
- **Exceptional senaryo:** User A yukarı düğmesine basar ama asansör gelince *aşağı* gitmek istediğini fark eder ve yanlış asansöre biner vb.

7.3.2 Entity class modeling: isim çıkarma (noun extraction)

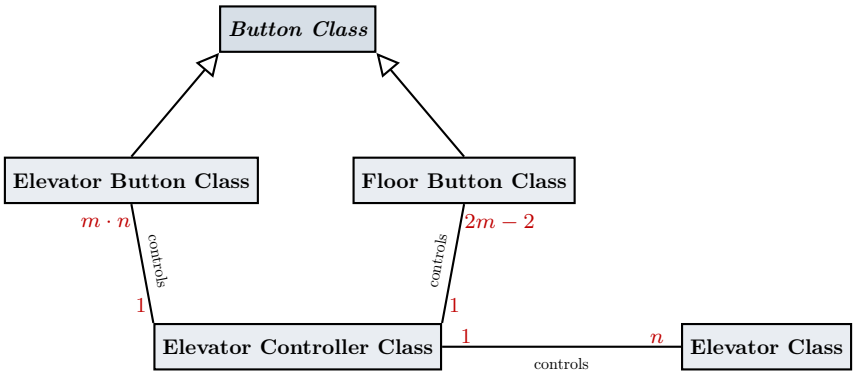
Sınıfları çıkarmanın yolları: senaryolardan çıkarmak (çok senaryo ⇒ çok aday sınıf tehlikesi), **CRC kartları** (alan bilgisi varsa), **noun extraction**. İsim çıkarma iki aşamalıdır:

1. **Kısa problem tanımı:** ürünü **tek paragrafta** tanımla.

   Buttons in elevators and on the floors control the movement of n elevators in a building with m floors. Buttons illuminate when pressed to request the elevator to stop at a specific floor; the illumination is canceled when the request has been satisfied. When an elevator has no requests, it remains at its current floor with its doors closed.

2. **İsimleri belirle** ve aday sınıf olarak kullan:
   - İsimler: *button, elevator, floor, movement, building, illumination, request, door.*
   - *floor, building, door* problem sınırının **dışında** ⇒ çıkar.
   - *movement, illumination, request* **soyut isimler** ⇒ çıkar (nitelik olabilirler).
   - Aday sınıflar: **Elevator Class** ve **Button Class**; alt sınıflar: **Elevator Button Class** ve **Floor Button Class**.

Sorun: düğmeler asansörlerle doğrudan iletişim kurmaz ⇒ ek bir sınıf gerekir: **Elevator Controller Class**. Böylece bütün ilişkiler **1-to-n** olur; bu tasarımı ve gerçekleştirmeyi kolaylaştırır.



7.3.3 CRC kartları

1989'dan beri OOA'da kullanılır. Her sınıf için bir kart doldurulur:

CLASS: sınıfın adı
RESPONSIBILITY: sınıfın işlevleri (ne yapar, neyi bilir)
COLLABORATION: çağırdığı / birlikte çalıştığı sınıflar

Bugün CRC kartları CASE araçlarıyla otomatik üretilir. **Güçlü yanı:** ekip üyeleri kartları canlandırınca (act out) **eksik veya hatalı öğeleri** ortaya çıkarmada çok güçlüdür (aynı zamanda iyi bir **test** tekniğidir). **Zayıf yanı:** entity sınıflarını bulmak için kullanılırsa **alan uzmanlığı** gerekir. (WhatsApp: CRC kartlarında sınıflar oklarla birbirine bağlanmaz.)

7.3.4 Dynamic modeling: statechart

Her sınıf için bir **UML statechart** çizilir. Durum (state), olay (event) ve koşul (predicate) diyagrama dağıtılmıştır. Statechart, senaryoların olaylarını modelleyerek kurulur. Notasyon: **dolu siyah daire** = başlangıç durumu; **içinde küçük siyah daire olan daire** = bitiş durumu; **köşeleri yuvarlatılmış dikdörtgen** = durum; **ok** = durumdan duruma geçiş (üzerinde onu tetikleyen olay).

7.3.5 Test workflow: CRC ile hataların bulunması

Elevator Controller'ın CRC kartında bir sorumluluk: *"1. Turn on elevator button"*. Bu nesne yönelimli paradigmaya **tamamen uygunsuzdur**: **responsibility-driven design** ve **information hiding** göz ardı edilmiştir. Doğrusu: *"1. Send message to Elevator Button Class to turn itself on."* Ayrıca bir sınıf unutulmuştur: asansör **kapılarının** çalışma sırasında değişen bir durumu vardır ⇒ **Elevator Doors Class** eklenir (güvenlik açısından da önemli). Class diagram değişince use-case diagram (değişmez), statechart'lar ve senaryolar yeniden gözden geçirilir.

7.3.6 God class ve dağıtık mimari

Üçüncü yinelemede *Elevator Controller Class* her şeyi yönetir. Bu bir **God class**'tır: çok fazla bilgiye maruz kalan ve çok fazla kontrole sahip sınıf; iyi bilinen bir **antipattern**'dir. Çözüm: **kontrolü dağıtmak (distributed decentralized architecture)**: her asansörün kendi *Elevator Subcontroller*'ı, her katın kendi *Floor Subcontroller*'ı vardır; $m+n$ alt kontrolcü istekleri işleyen bir *Scheduler* ile iletişim kurar; her katın üstünde ve altında bir *Sensor* asansör yaklaşınca/uzaklaşınca ilgili alt kontrolcüyü bilgilendirir. (Hoca: *tek bir sınıfa sorumluluk yüklemek yerine sorumlulukları dağıtmak.*) Analiz "şimdilik" doğrudur; tasarımda tekrar dönülebilir.

7.4 Boundary ve control sınıflarını çıkarmak

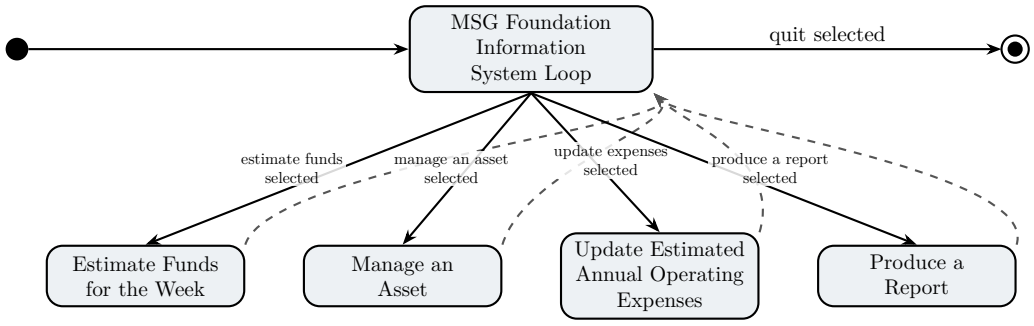
Boundary sınıflarını bulmak genellikle kolaydır: **her giriş ekranı, çıkış ekranı ve basılı rapor** bir boundary sınıfıdır. Her **hesaplama** genellikle bir control sınıfıdır.

7.5 MSG Foundation vaka çalışması

7.5.1 İlk sınıf diyagramı (noun extraction)

Tek paragraf: *Weekly reports are to be printed showing how much money is available for mortgages. In addition, lists of investments and mortgages must be printed on demand.* İsimler: *report, money, mortgage, list, investment. report* ve *list* uzun ömürlü değil (report bir boundary sınıfı olacak); *money* soyut isim. Aday entity sınıfları: **Mortgage Class** ve **Investment Class**. İkisine yapılan işlemler çok benzer (ekleme, silme, değiştirme, istenince yazdırma) ⇒ ortak üst sınıf **Asset Class**; *Manage a Mortgage* ve *Manage an Investment* birleşip **Manage an Asset** olur. Durum diyagramı kontrol edilince yıllık işletme giderlerinin saklanacağı bir sınıf olmadığı görülür ⇒ **MSG Application Class** eklenir ("iterate and increment" bazen *decrement* demektir).

7.5.2 İlk dinamik model



Dolu siyah daire başlangıç, içi noktalı daire bitiş durumudur. MSG personeli menüden beş komuttan birini seçer; her seçim bir **olaydır** ve bir durum geçişine neden olur. İşlem bitince sistem döngü durumuna geri döner.

7.5.3 Boundary ve control sınıfları

- Dört use case (Estimate Funds Available for Week, Manage an Asset, Update Estimated Annual Operating Expenses, Produce a Report) için tek ekran yeter ⇒ **User Interface Class**.
- Üç rapor ⇒ üç boundary sınıfı: **Estimated Funds Report Class**, **Mortgages Report Class**, **Investments Report Class**.
- Tek hesaplama ⇒ tek control sınıfı: **Estimate Funds for Week Class**.

7.6 Use-case realization

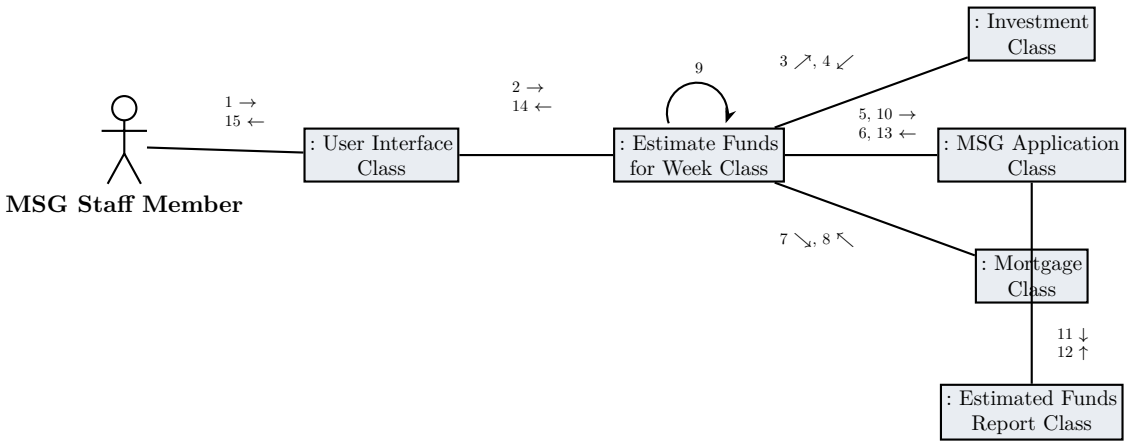
Use case'leri genişletip inceltme sürecine **use-case realization** denir (*realize* burada "gerçekleştirmek, başarmak" anlamındadır). Bir use case'in belirli bir senaryosunun gerçekleştirilmesi bir **interaction diagram** ile gösterilir: ya **sequence diagram** ya da **collaboration (communication) diagram**. Çalışan bir sistem sınıfları değil **nesneleri** kullanır: belirli bir ipotek `: Mortgage Class` biçiminde gösterilir.

Estimate Funds Available for Week use case'i için birbirinden türeyen artifact zinciri:

1. **Use case** (diyagram): aktör ile ürün arasındaki bütün olası etkileşimler.
2. **Use case açıklaması**: use case'in yazılı anlatımı.
3. **Class diagram**: use case'i gerçekleştiren sınıflar ve ilişkileri (nesneleri ve mesaj sırasını göstermez).
4. **Scenario**: use case'in belirli bir örneği.
5. **Collaboration diagram**: o senaryonun gerçekleştirilmesinde nesneler ve **gönderilme sırasına göre numaralanmış** mesajlar.
6. **Flow of events**: collaboration diyagramının yazılı anlatımı (müşteri anlamadan spesifikasyonu onaylamaz).
7. **Sequence diagram**: collaboration diyagramıyla **tamamen eşdeğer**.

Bu use case'e giren altı sınıf: *User Interface Class* (boundary), *Estimate Funds for Week Class* (control), *Mortgage Class*, *Investment Class*, *MSG Application Class* (entity), *Estimated Funds Report Class* (boundary).

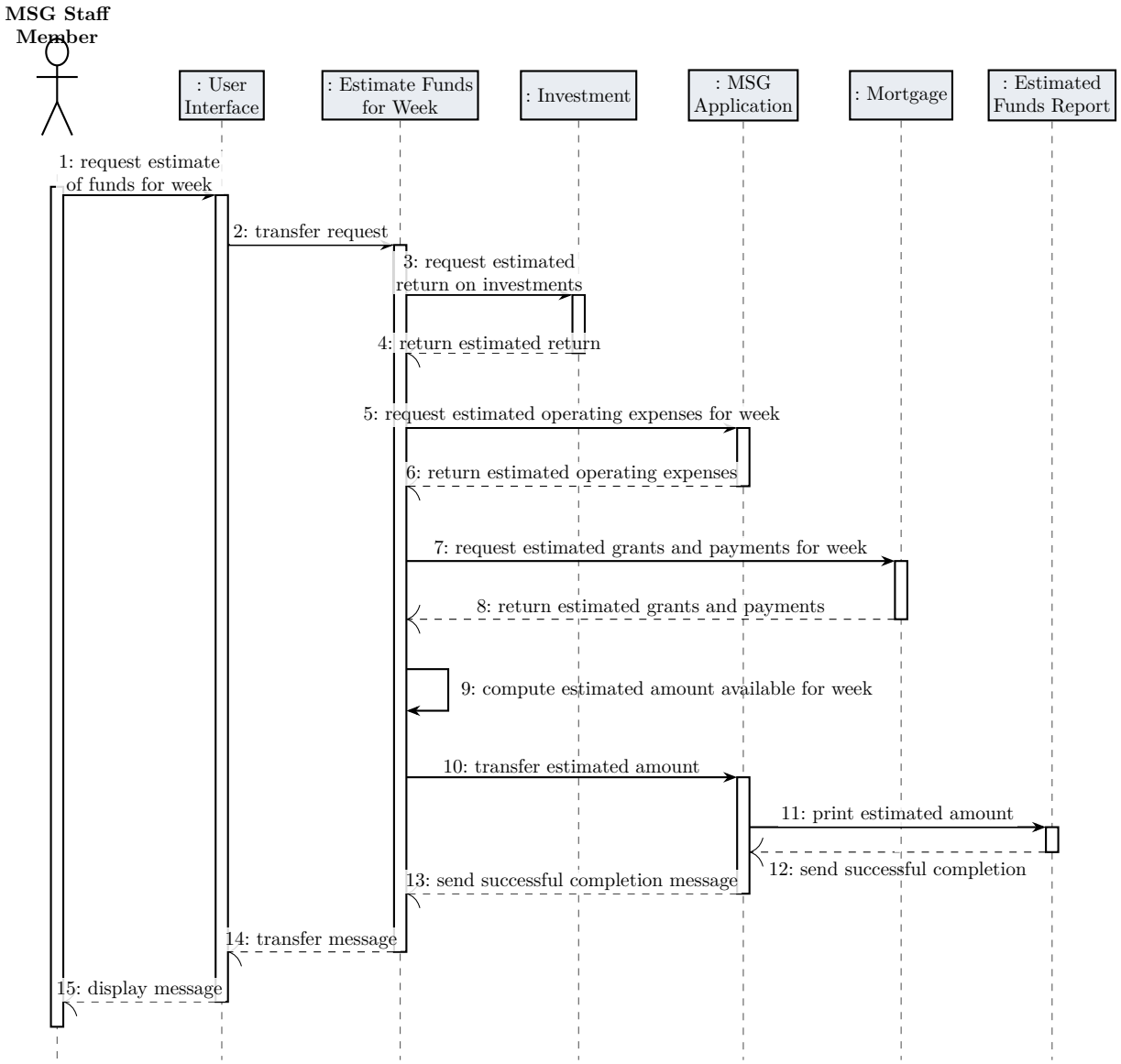
7.6.1 Collaboration diagram



Collaboration diyagramı nesneleri ve aralarındaki **bağlantıları (links)** gösterir; mesajlar bağlantıların üzerinde **numaralı** oklarla verilir. Dikey zaman ekseni yoktur. Mesajların listesi:

#	Gönderen	Alıcı	Mesaj
1	MSG Staff Member	: User Interface Class	Request estimate of funds available for week
2	: User Interface Class	: Estimate Funds for Week Class	Transfer request
3	: Estimate Funds for Week	: Investment Class	Request estimated return on investments for week
4	: Investment Class	: Estimate Funds for Week	Return estimated return on investments for week
5	: Estimate Funds for Week	: MSG Application Class	Request estimated operating expenses for week
6	: MSG Application Class	: Estimate Funds for Week	Return estimated operating expenses for week
7	: Estimate Funds for Week	: Mortgage Class	Request estimated grants and payments for week
8	: Mortgage Class	: Estimate Funds for Week	Return estimated grants and payments for week
9	: Estimate Funds for Week	(kendisi)	Compute estimated amount available for week (**self call**)
10	: Estimate Funds for Week	: MSG Application Class	Transfer estimated amount available for week
11	: MSG Application Class	: Estimated Funds Report Class	Print estimated amount available
12	: Estimated Funds Report	: MSG Application Class	Send successful completion message
13	: MSG Application Class	: Estimate Funds for Week	Send successful completion message
14	: Estimate Funds for Week	: User Interface Class	Transfer successful completion message
15	: User Interface Class	MSG Staff Member	Display successful completion message

7.6.2 Eşdeğer sequence diagram



Sequence diyagramının notasyonu (UML sunumu): aktör ve nesneler soldan sağa dizilir; **zaman yukarıdan aşağı** akar; her nesnenin altında kesikli **hayat çizgisi (lifeline)**; nesne çalışırken ince dikdörtgen **aktivasyon (activation)** kutusu; **çağrılar düz çizgili ok**, **dönüşler kesikli çizgili ok**; nesnenin kendine gönderdiği mesaj (**self call**) kendi hayat çizgisine dönen oktur.

7.6.3 Sequence mı, collaboration mı?

- **Sequence diagram**'ın gücü: mesajların akışını ve **sırasını** açık ve kesin gösterir. Odak **bilgi aktarımı** ise sequence diyagramı üstündür.
- **Collaboration diagram** bir sınıf diyagramına benzer; geliştiriciler **sınıflara** odaklanıyorsa daha kullanışlıdır.
- İkisine birlikte **interaction diagram** denir ve birbirine dönüştürülebilir.

Sequence diyagramı çizerken

- Analiz aşamasında (kitaptaki MSG diyagramları) metotlar henüz belli olmadığı için oklara *"Request estimated return..."* gibi açıklamalar yazılır; hoca: *"Dizayn aşamasına geçip metotlar belli olduktan sonra buralara metotların isimlerini yazmamız gerekecek."* Sınavda (sınıfları sizin tasarladığınız sorularda) metot adlarını yazın.
- Her okun üzerine yazılan çağrı, **okun ucundaki sınıfın bir metodu** olmalıdır (ör. : `Mortgage`'a giden ok `getGrantsAndPayments()`). Kesikli dönüş oklarına dönen değer yazılır.
- Akış use case açıklamasındaki adımlarla birebir uyuşmalı; aktör → boundary → control → entity sırası iyi bir şablondur.
- Koşullu akışlar (if/else) sorulabilir: UML'de `alt` kutusu (combined fragment) ile "[stok var]" / "[stok yok]" bölmeleri çizilir.

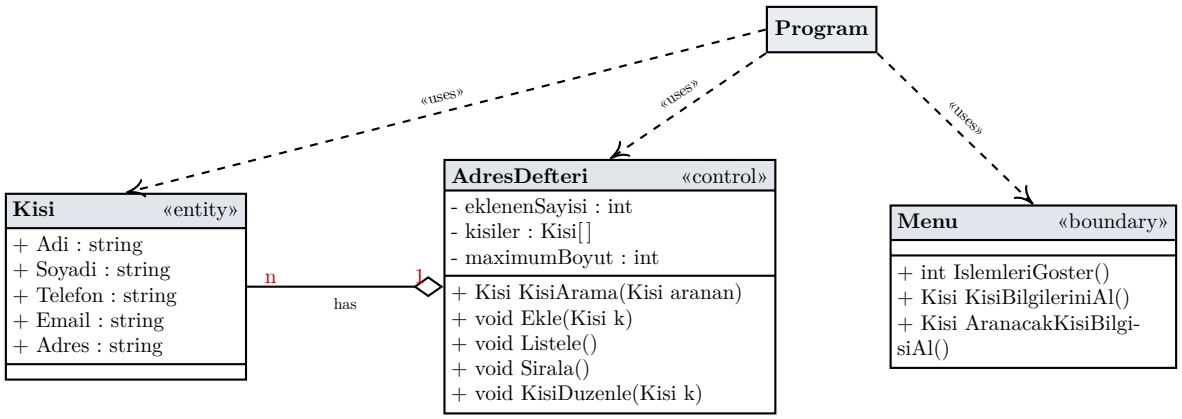
7.7 Hocanın tam çözümlü örneği: Adres Defteri

! Hoca derste vurguladı Kaynak: `Sunular/diagramlar.jpg` ve tahta fotoğrafı

Hocanın kendi el yazısıyla hazırladığı "Bahriye Akay – Nesne Yönelimli Prog. – Adres Defteri Örneği" kâğıdı: use case diyagramı, *Kişi Ekle* açıklaması, sınıf diyagramı (entity/control/boundary), CRC kartları ve *Kişi Ekle* collaboration diyagramı. Notlar: *"tüm usecase'ler için"*, *"tüm işlemler için yapılır"*, *"tüm sınıflar için"*. Yani sınavda senaryo verilirse **bütün** use case'lerin açıklaması, **bütün** sınıfların CRC kartı istenebilir. Use case diyagramı ve açıklamalar için Konu 6, Bölüm 6.6'ya bakın.

Kişi Ekle (hocanın kâğıdındaki hâli): *Kısa tanım:* eklenecek bilgilerin kaydı. *Adım adım tanım:* 1. Menüden ekleme seçilir. 2. Kişi bilgileri alınır. 3. Maksimum boyut aşılmamışsa eklenir. (Use case'ler: Kişi Ekle, Kişi Düzenle, Arama, Kişi Sil, Listele, Sırala, Çıkış; *Kapasite Dolu* ve *İptal* «extends»; Düzenle ve Sil «include» Arama.)

7.7.1 Sınıf diyagramı



- **Kisi** uzun ömürlü bilgiyi tutar ⇒ entity. **Menu** kullanıcıdan bilgi alır, menüyü gösterir ⇒ boundary. **AdresDefteri** kişiler üzerinde ekleme, arama, sıralama işlemlerini yürütür; hoca kâğıtta bunu **control** olarak işaretledi.
- **AdresDefteri ◊— Kisi: aggregation** (*"AdresDefteri has Kişi(ler)"*): bir adres defteri n kişiden oluşan bir topluluk tutar. Elmas AdresDefteri tarafında.
- **Program** diğer sınıfları kullanır (*uses* = dependency).

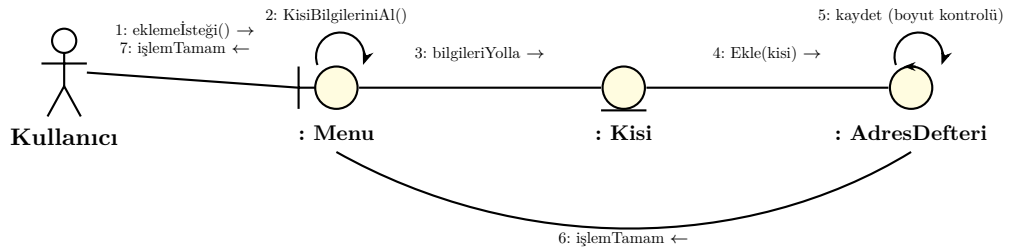
- Hocanın tahtadaki (daha yeni) sürümünde ise kontrol ayrı bir **DefterController** sınıfına alınmış, AdresDefteri kişileri tutan sınıf olarak kalmış, kullanıcı arayüzü **KullaniciArayuzu** (AnaMenuUret, EkleMenu, SilMenu …) ve **Raporlama** (printRapor) boundary sınıfları eklenmiş; Kisi ile AdresDefteri arası **dolu elmas (composition)** çizilmiş. İki çözüm de geçerlidir: ilişki türünü **gerekçesiyle** yazın (kişiler defterle birlikte silinir ⇒ composition; kişi defterden bağımsız yaşayabilir ⇒ aggregation).

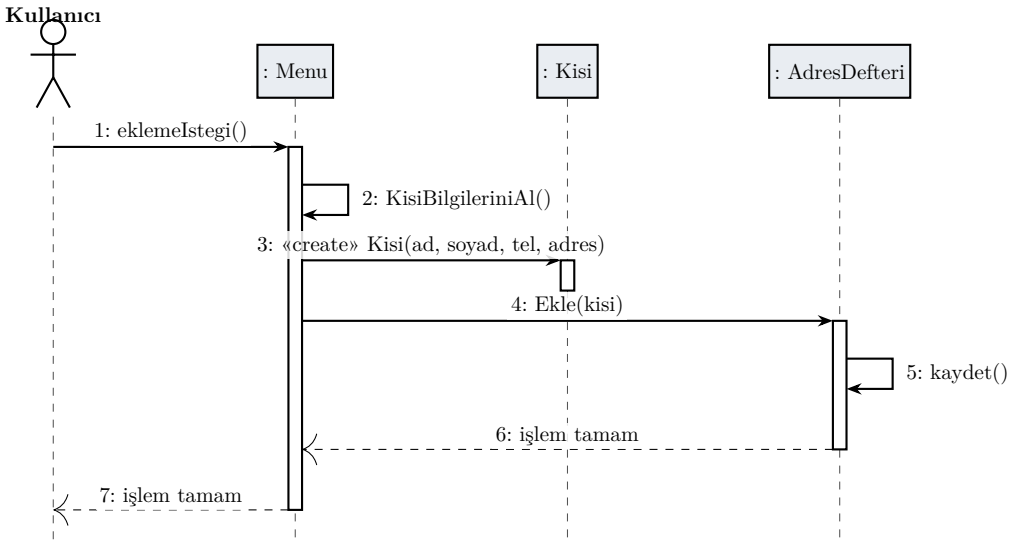
7.7.2 CRC kartları (hocanın kâğıdından)

CLASS: AdresDefteri	**CLASS: Menu**	**CLASS: Program**
Responsibility: kişi ekler; kişi siler; kişi düzenler; arama yapar; sıralama yapar	**Responsibility:** ana menüyü gösterir; kişi ekleme bilgilerini alır; kişi arama bilgisini alır	**Responsibility:** menüyü listelemek için Menu nesnesine mesaj yollar; işlemler için AdresDefteri'ne mesaj yollar
Collaboration: Kisi, Program	**Collaboration:** Kisi, AdresDefteri, Program	**Collaboration:** Kisi, Menu, AdresDefteri

Kisi için de: *Responsibility:* kişi bilgilerini tutar ve verir; *Collaboration:* AdresDefteri.

7.7.3 Kişi Ekle: collaboration ve sequence diyagramı





Akış: kullanıcı ekleme ister (1), menü kişi bilgilerini alır (2, self call), bilgilerle Kisi nesnesi oluşturulur (3), kişi adres defterine eklenmek üzere gönderilir (4), adres defteri maksimum boyutu kontrol edip kaydeder (5, self call), sonuç geriye bildirilir (6, 7). Kapasite doluysa *Kapasite Dolu* extend akışı çalışır (`alt` çerçevesiyle gösterilebilir).

7.8 Diğer konular

- **Specification document in the Unified Process:** Unified Process **use-case driven**'dır; use case'ler ve onlardan türeyen artifact'lar geleneksel metin spesifikasyonunun yerini alır ve aynı sözleşme rolünü oynar. Müşteri ürünün nasıl çalışacağını use case + senaryolardan da rapid prototype'tan da anlayabilir; fark: use case'ler her seferinde inceltilir, rapid prototype atılır. Yine de kullanıcı arayüzünün prototipi (örnek ekranlar, raporlar) gerekir.
- **Aktörleri ve use case'leri bulmak:** bir kişinin ürünle etkileşebileceği her rolü düşün; aktörler bireyler değil **rollerdir**. Unified Process rol için *worker* terimini kullanır (Applicants ve Borrowers iki ayrı worker). İş modelinin yazılıma karşılık gelen alt kümesindeki aktörler, aradığımız aktörlerdir; her rol için bir veya daha fazla use case vardır.
- **OOA'nın zorlukları:** nesne yönelimli tasarımın sınırını geçme; **sınıflara henüz metot atama** (stepwise refinement sırasında metotları yeniden dağıtmak boşa emektir).
- **Metrikler:** size, cost, duration, effort, quality; OOA'nın boyut ölçüsü: UML diyagramlarının sayfa sayısı.

7.9 Kısa tanım soruları

Entity / boundary / control class Models long-lived information / models the interaction between the product and its environment (input, output, reports) / models complex computations and algorithms.

Scenario An instance of a use case.

Noun extraction Describe the product in a single paragraph, identify the nouns, discard those outside the problem boundary and the abstract nouns; the rest are candidate classes.

CRC card Class–Responsibility–Collaboration card: the name of the class, its responsibilities and the classes it invokes.

God class A class that is exposed to too much information and has too much control (an antipattern).

Use-case realization The process of extending and refining use cases; the realization of a scenario is depicted by a sequence or collaboration diagram.

Collaboration diagram Shows the objects, the links between them and the messages numbered in the order in which they are sent.

Sequence diagram Shows the objects and the messages between them in time order (top to bottom); calls are solid arrows, returns dashed arrows.